

SANDS ORNAMENTS

Apple App Store Publishing — Readiness & Implementation Report

Full module analysis · Exact change list · Submission checklist

Prepared: 11 June 2026

Project: Sands Ornaments (React + Vite frontend, Node/Express + MongoDB backend)

Status: Already live on Google Play Store — this document covers iOS only

Contents

1. Executive Summary
2. Current Project Analysis (Tech Stack & Modules)
3. Module-by-Module iOS Compatibility Analysis
4. Key Decision: What to Ship in the iOS App
5. Accounts, Hardware & Cost Requirements
6. Exact Code Changes Required (Implementation List)
7. iOS Project Configuration (Info.plist, Privacy Manifest)
8. App Store Connect Submission Checklist
9. Apple Review — Rejection Risks & Mitigations
10. What Does NOT Need to Change
11. Step-by-Step Roadmap & Timeline

1. Executive Summary

Sands Ornaments is a jewellery e-commerce web application (React 19 + Vite) already published on the Google Play Store, most likely as a wrapped web app. Publishing to the Apple App Store requires wrapping the same web frontend with **Capacitor**, because Apple has no equivalent of Android's Trusted Web Activity and rejects plain website shells.

Three critical findings:

1. **Push notifications will break on iOS.** The current implementation (`pushNotificationService.js`) uses Firebase *Web Push* with a service worker (`firebase-messaging-sw.js`). Service-worker web push does not work inside an iOS app `WebView`. It must be replaced with the native Capacitor push plugin + APNs. This is the single biggest required code change.
2. **Apple Guideline 4.2 (Minimum Functionality) risk.** Apple rejects apps that are "just a website in a box." Native push notifications + native camera/scanner + splash/status-bar integration are needed to pass.
3. **Login is phone-OTP via SMS.** Apple reviewers cannot receive your SMS. A test phone number with a fixed OTP must be provided, or the review will fail at the login screen (Guideline 2.1).

Good news already verified in the code: auth uses Bearer tokens in `localStorage` (no cookie problems in a native shell); in-app **account deletion already exists** (backend `deleteAccount` + Profile page) which satisfies Apple Guideline 5.1.1(v); and Razorpay is fine because you sell **physical goods** — Apple In-App Purchase is not required.

2. Current Project Analysis

2.1 Tech Stack

Layer	Technology	iOS Relevance
Frontend	React 19, Vite 7, Tailwind CSS 4, React Router 7, TanStack Query, Framer Motion, GSAP, Lenis	Wraps cleanly with Capacitor; bundled into the app via <code>dist/</code>
Backend	Node.js / Express, MongoDB (Mongoose), JWT auth, deployed behind Vercel config	No changes required except one small addition (review test OTP)
Payments	Razorpay (web checkout, <code>window.Razorpay</code>), refunds implemented	Allowed (physical goods); needs WebView testing + UPI scheme config
Push	Firebase Cloud Messaging — Web Push + service worker	Must be replaced on iOS
Auth	User: phone OTP (SMS). Seller: email/password + reset OTP. Admin: password	Works; reviewer test account needed
Media	Cloudinary uploads, html5-qrcode camera scanner (seller)	Needs iOS camera/photo permission strings
Realtime	Socket.io client	Works in Capacitor unchanged
Shipping	Delhivery integration (backend)	No change

2.2 Module Inventory

Module	Contents (from code analysis)
User storefront (~43 pages)	Home, Shop (Men / Women / Family), Gold Jewellery, Collections (Bond, Best Styles, Vault, Catalog), Product Details, Cart, Checkout (Razorpay), Wishlist, Order Success / Tracking, Returns, Replacements, Gift Cards, Offers, Blogs, Notifications, Profile (addresses, payments, account deletion), Login (OTP), CMS pages (About, Privacy Policy, Terms, Help Center, dynamic pages)
Seller portal (~28 pages)	Dashboard, Analytics, Inventory, Products + Product Editor, Product Barcodes, QR Scanner (camera) , Offline Sale / Direct Sales (POS), Orders, Shipments, Pickup Locations, Returns, Replacements, Customers, Commission, Metal Pricing, Notifications, Profile, Registration with KYC document upload (Aadhaar, shop license, certificate)
Admin panel	Dashboard, Orders (incl. refunds), Products, Inventory, Users, Sellers, Reviews, Returns/Replacements, Coupons, Banners, Blogs, CMS pages, FAQ, Support tickets, Settings, Notifications, Audit log
Backend modules	<code>auth</code> (user/seller/admin), <code>user</code> , <code>seller</code> , <code>admin</code> , <code>public</code> , <code>shared</code> ; services: Cloudinary, Delhivery, SMS (OTP), push; middlewares: authenticate, rate limiters, upload, error handling

3. Module-by-Module iOS Compatibility Analysis

Feature / Module	Status	Analysis & Required Action
User login (phone OTP)	Action needed	Works technically. Add a whitelisted test phone number with a fixed OTP in the backend (e.g. +91 9999999999 → OTP 123456, enabled by env flag) so Apple reviewers can log in.
Token storage (localStorage Bearer JWT)	Works	Bearer-header auth works in Capacitor's WebView. Optional hardening later: move tokens to <code>@capacitor/preferences</code> or secure storage.
Account deletion	Already done	Backend <code>deleteAccount</code> cascades user data (addresses, notifications, OTPs...); exposed in Profile UI. Satisfies Apple 5.1.1(v). Just verify it is reachable within 2–3 taps in the app.
Checkout — Razorpay web checkout	Action needed	<code>new window.Razorpay(options)</code> opens inside the WebView. Card/netbanking flows generally work; UPI intent flows (GPay/PhonePe/Paytm app jumps) need <code>LSApplicationQueriesSchemes</code> in Info.plist and end-to-end testing on a real iPhone. Since goods are physical, Apple IAP rules do <u>not</u> apply (Guideline 3.1.3(e)).
Push notifications	Will break	Web Push service worker is not available inside the iOS WebView. Replace with <code>@capacitor/push-notifications</code> behind a platform check; web keeps its current path. Backend keeps sending via FCM — FCM routes to APNs automatically once the APNs key (.p8) is uploaded to the Firebase console.
QR scanner (seller, html5-qrcode)	Action needed	<code>getUserMedia</code> works in iOS WKWebView (iOS 14.3+) but only with <code>NSCameraUsageDescription</code> set. For reliability, the native <code>@capacitor-mlkit/barcode-scanning</code> plugin is recommended. (Only relevant if the seller portal ships in the iOS app — see Section 4.)
File uploads (KYC docs, product images)	Action needed	<code><input type="file"></code> works in WKWebView, but iOS shows camera/photo-library options → photo library and camera permission strings are mandatory or the app crashes on tap.
Socket.io realtime	Works	WebSocket connections work unchanged from the Capacitor WebView.
React Router navigation	Works	Browser-history routing works when assets are bundled. Must verify no hard-coded absolute URLs to the website domain for internal navigation.
Images via Cloudinary / animations (GSAP, Framer, Lenis)	Works	Remote images load fine. Lenis smooth-scroll should be disabled on native (iOS momentum scrolling conflicts) — small platform check.
UI fit on iPhone (notch / home bar)	Action needed	Add <code>viewport-fit=cover</code> + <code>env(safe-area-inset-*)</code> padding for header/bottom nav; disable pinch-zoom and overscroll bounce for a native feel.

External links (Privacy, Terms, blogs sharing)	Minor	External links should open via <code>@capacitor/browser</code> (in-app Safari sheet) instead of navigating the WebView away from the app.
Admin panel	Exclude	Should not ship inside the iOS app (Apple dislikes hidden admin surfaces; it adds review risk and no user value). Keep it web-only.

4. Key Decision: What to Ship in the iOS App

	Option A — Customer app only Recommended	Option B — Customer + Seller in one app
Scope	User storefront only; seller & admin stay on the web	Storefront + seller portal (dashboard, scanner, POS)
Review risk	Lower — one clear consumer purpose (Shopping)	Higher — reviewer sees seller registration w/ KYC uploads; more screens to break
Native work	Push + splash/status bar + safe areas	All of A + camera scanner plugin + upload permissions
Effort	Smaller, faster to approval	+30–40% effort and testing

Recommendation: ship Option A first (matches what most customers need), keep sellers on the mobile website, and add the seller module in version 1.1 once the app is approved and stable. The change list in Section 6 marks seller-only items separately.

5. Accounts, Hardware & Cost Requirements

Item	Details	Cost
Apple Developer Program	Enroll at developer.apple.com. For a business (organization) account you need a D-U-N-S number (free, takes up to ~2 weeks in India if not already issued). Individual account is faster but shows your personal name as seller.	US\$99 / year (recurring)
Mac with Xcode 16+	Required to build, sign and upload. Alternatives if no Mac: Codemagic CI (has free tier, builds + uploads Capacitor iOS apps) or a rented cloud Mac (MacinCloud).	₹0 (own Mac) / Codemagic free tier or ~\$0.04/min
iPhone for testing	Strongly recommended — Razorpay UPI flows and push must be tested on a real device (simulator has no push/UPI apps).	Any iPhone running iOS 16+
APNs key (.p8)	Created in the Apple Developer portal, uploaded once to Firebase Console → Project Settings → Cloud Messaging. Lets the existing FCM backend deliver to iPhones.	Free
Certificates / provisioning	Let Xcode "Automatically manage signing" — no manual work needed.	Free

6. Exact Code Changes Required

Numbered in recommended execution order. ★ = required for approval; ☆ = strongly recommended; (S) = only if the seller portal ships in the app.

6.1 ★ Add Capacitor to the frontend

- Install: `@capacitor/core` , `@capacitor/cli` , `@capacitor/ios`
- Create `capacitor.config.ts` — appld e.g. `com.sandsornaments.app` (must match Play Store package style but can differ), `appName` "Sands Ornaments", `webDir`: "dist"
- Workflow: `npm run build` → `npx cap sync ios` → open/build in Xcode (or Codemagic)
- Web assets are **bundled inside the app** (do not use a remote `server.url` pointing at the website — Apple treats that as a thin wrapper and it worsens the 4.2 risk)

6.2 ★ Replace push notifications on native (biggest change)

- Install `@capacitor/push-notifications`
- In `pushNotificationService.js` : branch on `Capacitor.isNativePlatform()` —
 - **Native:** request permission via the plugin, get the device token from the `registration` event, send it to the existing backend token-registration endpoint (same API the web token uses today), handle `pushNotificationReceived` / `pushNotificationActionPerformed` for foreground display and tap-navigation
 - **Web:** keep the current service-worker path unchanged
- Xcode: enable **Push Notifications** capability + **Background Modes** → **Remote notifications**
- Firebase Console: upload the APNs .p8 key (one-time, no backend code change — FCM delivers to APNs automatically)
- Add `GoogleService-Info.plist` (iOS app registered in the same Firebase project) and the Firebase iOS SDK via the Capacitor plugin's documented setup, OR use FCM's APNs token mapping per plugin docs

6.3 ★ Reviewer test login (backend, small)

- In `userAuth.controller.js` `sendOtp` / `verifyOtp` : support an env-configured test number (e.g. `REVIEW_TEST_PHONE` , `REVIEW_TEST_OTP`) that skips the SMS send and accepts the fixed OTP. Keep it disabled unless the env vars are set.

6.4 ★ iOS UI fit & native feel

- `index.html` : add `viewport-fit=cover` ; CSS: pad fixed header / bottom nav / toasts with `env(safe-area-inset-top/bottom)`
- Disable pinch-zoom (`maximum-scale=1`) and overscroll bounce (`overscroll-behavior: none`)
- Platform check to disable Lenis smooth-scroll on native (let iOS momentum scrolling work naturally)

- Add `@capacitor/status-bar` (style/colour matching the gold/dark theme) and `@capacitor/splash-screen`
- Handle the iOS keyboard overlapping inputs (Capacitor `Keyboard` plugin, `resize: body`)

6.5 ★ Razorpay on iOS

- Test the full `window.Razorpay` checkout in the WebView on a real iPhone (cards, netbanking, UPI)
- Add UPI app schemes to `LSApplicationQueriesSchemes` (Section 7) so intent buttons (GPay/PhonePe/Paytm) can open
- Verify the post-payment redirect/handler returns control to the app (Razorpay JS callback already used in `CartContext.jsx` — confirm it fires inside WKWebView)
- Fallback if UPI intents misbehave: open checkout via `@capacitor/browser` and confirm order status by polling the backend

6.6 ☆ External links & misc

- Open external URLs (social, courier tracking, blog share targets) with `@capacitor/browser`
- Hide any "Download our app" banners / Play-Store links when running on iOS (Apple rejects links to other stores)
- Add `@capacitor/app` to handle the Android-style back behaviour gracefully and app-state resume (refresh cart/notifications on resume)
- Verify no UI text says "Android" or shows Play-only features

6.7 (S) Seller portal items — only if included

- QR scanner: keep `html5-qrcode` (works on iOS 14.3+ with camera permission) or switch to `@capacitor-mlkit/barcode-scanning` for reliability
- KYC / product image uploads: camera + photo library permission strings (Section 7) — mandatory, otherwise tapping upload crashes the app
- Barcode printing pages (react-barcode) — verify rendering/printing expectations on iOS (no direct printing; offer share/save as image)

6.8 ★ App icon & splash assets

- 1024×1024 master icon (no transparency, no rounded corners — Apple rounds it) + splash source image; generate all sizes with `@capacitor/assets`

7. iOS Project Configuration

7.1 Info.plist entries

Key	Value / Purpose
NSCameraUsageDescription	"Sands Ornaments uses the camera to scan product QR codes and capture document photos." (S — required if seller portal or any camera use ships)
NSPhotoLibraryUsageDescription	"Allows you to choose photos for uploads." (needed for any <code><input type=file></code>)
NSPhotoLibraryAddUsageDescription	Needed if the app saves images (e.g. barcodes) to the library
LSApplicationQueriesSchemes	<code>tez</code> , <code>phonepe</code> , <code>paytmmp</code> , <code>upi</code> , <code>bhim</code> , <code>credpay</code> — lets Razorpay UPI intent buttons open the apps
ITSAppUsesNonExemptEncryption	NO — app only uses standard HTTPS; skips the export-compliance questionnaire on every upload
UIViewControllerBasedStatusBarAppearance	Per Capacitor status-bar plugin docs

7.2 Privacy manifest (PrivacyInfo.xcprivacy)

Required since 2024. Recent Firebase iOS SDKs ship their own manifests; the app target still needs one declaring "required reason" API use (typically `UserDefaults` → reason `CA92.1` via Capacitor). Capacitor 6+ templates include a starter file — verify it exists after `npx cap add ios` .

7.3 Capabilities (Xcode → Signing & Capabilities)

- Push Notifications
- Background Modes → Remote notifications
- (Optional, later) Associated Domains — for universal links like `https://sandsornaments.com/product/...` opening the app

8. App Store Connect Submission Checklist

#	Item	Notes for Sands
1	Create app record	Bundle ID = the Capacitor appId; Name "Sands Ornaments"; primary language English (India); Category: Shopping
2	Pricing	Free

3	Screenshots	Mandatory: 6.9"/6.7" iPhone — 1320×2868 or 1290×2796 px (3–10 images). Make the app iPhone-only in Xcode (uncheck iPad) to skip iPad screenshots. Suggested shots: Home, Shop, Product detail, Cart/Checkout, Order tracking, Profile
4	App icon	1024×1024, no alpha
5	Description / subtitle / keywords	Reuse Play Store listing text; keywords field is 100 chars (comma-separated, no spaces wasted)
6	Support URL + Privacy Policy URL	Privacy Policy page already exists in the app — must also be a public web URL
7	App Privacy (nutrition labels)	Declare data collected & linked to identity: Name, Phone number, Email, Physical address (shipping), Purchase history, Device token (push), User ID . Usage: App Functionality. "Used for tracking" = No (no ad SDKs found in the code). If you later add analytics/ads, this must be updated.
8	Age rating questionnaire	All "No" → rating 4+
9	Demo account for review	App Review Information → provide the test phone number + fixed OTP from change 6.3, plus a short note: "Login is via SMS OTP; use this test number, OTP is always 123456. We sell physical jewellery; payments via Razorpay."
10	Export compliance	Handled by <code>ITSAAppUsesNonExemptEncryption=NO</code>
11	Content rights / IDFA	No third-party content issues; IDFA = not used
12	TestFlight	Upload build, internal test on a real iPhone (push, OTP login, full purchase + refund flow) before submitting

9. Apple Review — Rejection Risks & Mitigations

Guideline	Risk for Sands	Mitigation
4.2 Minimum Functionality	High — web-wrapped shopping apps are the classic 4.2 rejection	Bundle assets locally (no remote URL shell); native push; splash + status bar + safe areas; native-feeling scroll; mention native capabilities in review notes
2.1 App Completeness	High if reviewer cannot log in (SMS OTP) or any page errors	Test OTP account (6.3); click through every page on a real device; no dead links/coming-soon screens (review the "Gold Coming Soon" page — either hide it on iOS or ensure it looks intentional)
5.1.1(v) Account deletion	Low — already implemented	Confirm visible in Profile within a few taps
3.1.3(e) Physical goods	Low	Jewellery = physical goods → Razorpay allowed, no IAP. Note: if Gift Cards are redeemable only inside the app for digital value, review carefully — gift cards for physical-goods purchases are fine
5.1.1 Data collection	Medium	Privacy labels (8.7) must match reality; ask for camera/photos permissions only at the moment of use
4.0 Design	Medium	No pinch-zoomed-out desktop layouts; tap targets adequate; keyboard does not cover inputs
3.1.1 / external purchase links	Medium	Remove any Play Store links/banners on iOS

Expect the first submission to take 1–3 days in review. A first-round rejection with questions is normal — respond in the Resolution Center, usually no re-development needed.

10. What Does NOT Need to Change

- **Backend** — all APIs, models, Razorpay refunds, Delhivery, Cloudinary, audit logs stay as-is (only the optional test-OTP whitelist is added)
- **Admin panel** — remains web-only
- **Auth approach** — phone OTP is acceptable to Apple; "Sign in with Apple" is NOT required because there is no third-party social login (verified: no Google/Facebook sign-in in the code)
- **Razorpay as payment provider** — permitted for physical goods
- **Play Store build** — untouched; iOS is an additive platform
- **Website** — keeps working exactly as today; web push path is preserved

11. Step-by-Step Roadmap & Timeline

Phase	Work	Duration
1. Accounts	Enroll in Apple Developer Program (start D-U-N-S immediately if organization account)	2 days – 2 weeks (parallel)
2. Capacitor setup	Changes 6.1, 6.4, 6.6, 6.8 — wrap, UI fit, assets	2–3 days
3. Push migration	Change 6.2 + APNs key + Firebase config	2–3 days
4. Payments & review prep	Changes 6.3, 6.5 — test OTP, Razorpay device testing	2 days
5. Build & TestFlight	Xcode/Codemagic build, full regression on a real iPhone	2–3 days
6. Store listing	Section 8 checklist, screenshots, privacy labels	1–2 days
7. Review	Submit; respond to reviewer if needed	1–5 days

Total realistic timeline: ~2–4 weeks, dominated by the Apple account approval and one possible review round-trip. Out-of-pocket cost: **\$99/yr** (+ optional CI minutes if no Mac).

Summary of files that will be touched when implementation starts (for reference — no changes made yet):

frontend/package.json (Capacitor deps) · frontend/capacitor.config.ts (new) · frontend/index.html (viewport) · frontend/src/services/pushNotificationService.js (platform branch) · frontend/src/context/CartContext.jsx (Razorpay verification only) · global CSS (safe areas) · frontend/ios/ (new native project: Info.plist, PrivacyInfo.xcprivacy, GoogleService-Info.plist, icons/splash) · backend/src/modules/auth/controllers/userAuth.controller.js (test OTP whitelist)